

# Appendix B: Numerical Methods

## Rigorous Treatment for Heterogeneous Agent Macroeconomics

Companion to hetmacro Codebook & Lecture Notes

This appendix provides a rigorous, macro-level treatment of the numerical building blocks used in solving heterogeneous-agent models: calibration and estimation, root finding, optimization, and Euler-equation methods. Examples and figures are drawn from Problem Set 1 (labor supply equilibrium, income process calibration, portfolio choice).

## 1 Calibration and Estimation

### 1.1 Method of Simulated Moments

Let  $\theta \in \Theta \subseteq \mathbb{R}^p$  denote the structural parameters and  $m(\theta)$  the vector of model-implied moments. We do not observe  $m(\theta)$  analytically; instead we approximate it by simulation. For a given  $\theta$ , simulate  $S$  paths of length  $T$ , compute sample moments  $\hat{m}_S(\theta)$ , and define the *moment residual*

$$g_S(\theta) = \hat{m}_S(\theta) - m_0, \quad (1)$$

where  $m_0$  is the vector of target (data) moments. The **method of simulated moments** (MSM) estimator is

$$\hat{\theta}_{\text{MSM}} = \arg \min_{\theta \in \Theta} g_S(\theta)^\top W g_S(\theta), \quad (2)$$

where  $W$  is a positive definite weight matrix. Common choices:  $W = I$  (equal weighting, sum of squared errors) or  $W = \hat{\Omega}^{-1}$  with  $\hat{\Omega}$  an estimate of the variance of the moment vector (optimal GMM weighting under regularity conditions).

**Objective function.** In practice we often use the sum of squared errors (SSE):

$$Q_S(\theta) = \|g_S(\theta)\|_2^2 = \sum_{j=1}^J (\hat{m}_{S,j}(\theta) - m_{0,j})^2. \quad (3)$$

Minimizing  $Q_S$  corresponds to  $W = I$ . The objective is generally non-smooth in  $\theta$  (because  $\hat{m}_S$  comes from finite simulation) and derivatives are unavailable or noisy; derivative-free optimizers such as Nelder–Mead are therefore standard.

## 1.2 Stochastic Process Specification

### 1.2.1 AR(1) with measurement error (baseline)

Let  $y_t$  denote observed log productivity (or log output) and  $x_t$  the persistent component. The process is

$$y_t = x_t + \varepsilon_t, \quad \varepsilon_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_\varepsilon^2), \quad (4)$$

$$x_t = \rho x_{t-1} + u_t, \quad u_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_u^2), \quad |\rho| < 1, \quad (5)$$

with  $\varepsilon_t$  and  $u_s$  independent for all  $t, s$ . The stationary variance of  $x_t$  is  $\text{Var}(x_t) = \sigma_u^2 / (1 - \rho^2)$ . Hence

$$\text{Var}(y_t) = \frac{\sigma_u^2}{1 - \rho^2} + \sigma_\varepsilon^2, \quad (6)$$

$$\text{Var}(\Delta y_t) = 2 \left( \frac{\sigma_u^2}{1 + \rho} + \sigma_\varepsilon^2 \right), \quad (7)$$

$$\text{Corr}(y_t, y_{t-k}) = \rho^k \cdot \frac{\sigma_u^2 / (1 - \rho^2)}{\sigma_u^2 / (1 - \rho^2) + \sigma_\varepsilon^2}, \quad k \geq 1. \quad (8)$$

### 1.2.2 Jump-diffusion extension (fat tails)

To capture fat tails in growth rates, extend the persistent component to a jump-diffusion:

$$x_t = \rho x_{t-1} + \eta_t + J_t, \quad \eta_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_\eta^2), \quad J_t = b_t \kappa_t, \quad (9)$$

with  $b_t \sim \text{Bernoulli}(\pi)$  and  $\kappa_t \sim \mathcal{N}(\mu_J, \sigma_J^2)$  independent of  $\eta_t$ . The jump term adds occasional large moves; moments generally have no closed form and must be computed by simulation.

## 1.3 Worked Example: Income Process Calibration

We target five moments:  $\text{sd}(y)$ ,  $\text{sd}(\Delta y)$ , and autocorrelations at lags 1, 3, and 5. For each candidate  $\theta = (\rho, \sigma_u, \sigma_\varepsilon)$  we simulate a panel of  $N$  firms over  $T$  periods, compute the five sample moments, and evaluate the SSE. Figure 1 shows the distribution of log-productivity changes at horizons 1, 3, and 5 from the simulated panel under the baseline AR(1)+noise specification; the densities are normal as implied by the model.

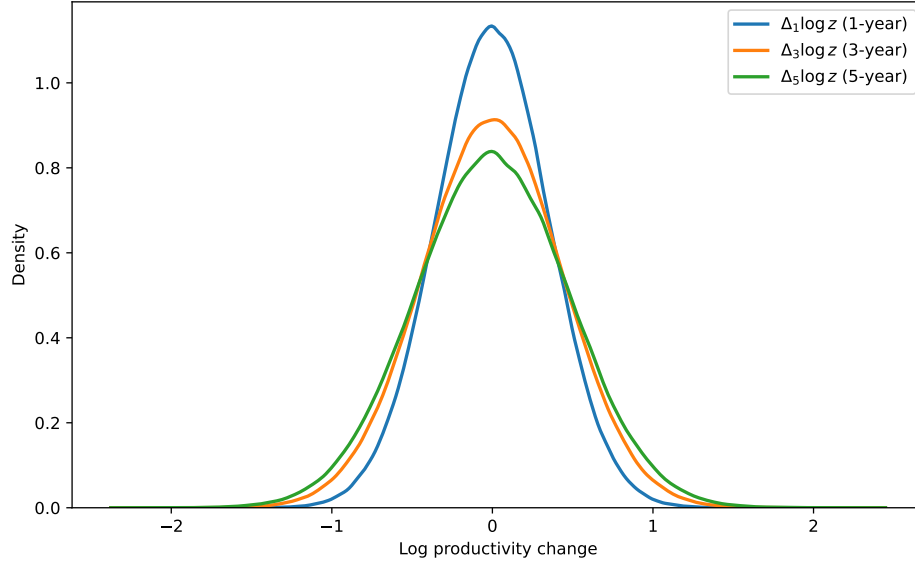


Figure 1: Distribution of log-productivity changes: density of  $\Delta_1 \log z$ ,  $\Delta_3 \log z$ , and  $\Delta_5 \log z$  from the simulated panel (baseline AR(1)+noise).

Figure 2 plots the sum of squared moment errors at each Nelder–Mead iteration. The algorithm converges to a low SSE; the calibrated parameters reproduce the target moments closely.

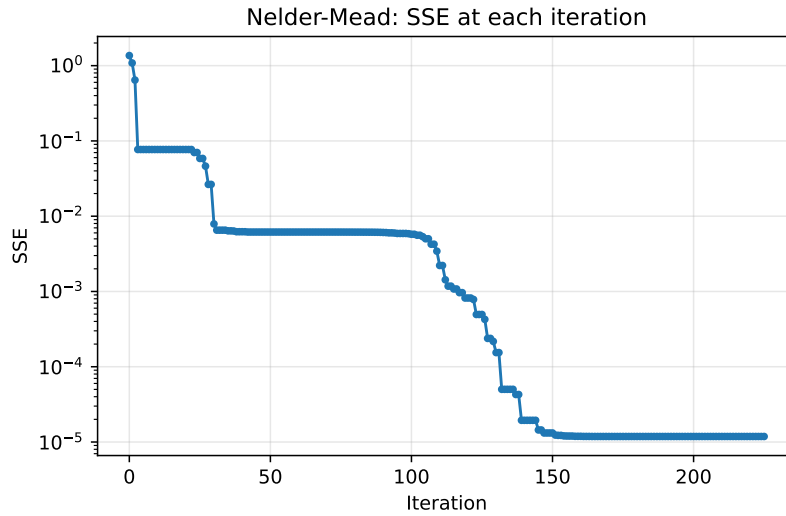


Figure 2: Nelder–Mead: SSE (sum of squared moment errors) at each iteration.

For the jump-diffusion extension, the objective has six parameters and no analytical moments. Figure 3 shows the Nelder–Mead SSE trace; Figure 4 shows the distribution of log-productivity changes under the jump-diffusion fit, with fatter tails than the baseline.

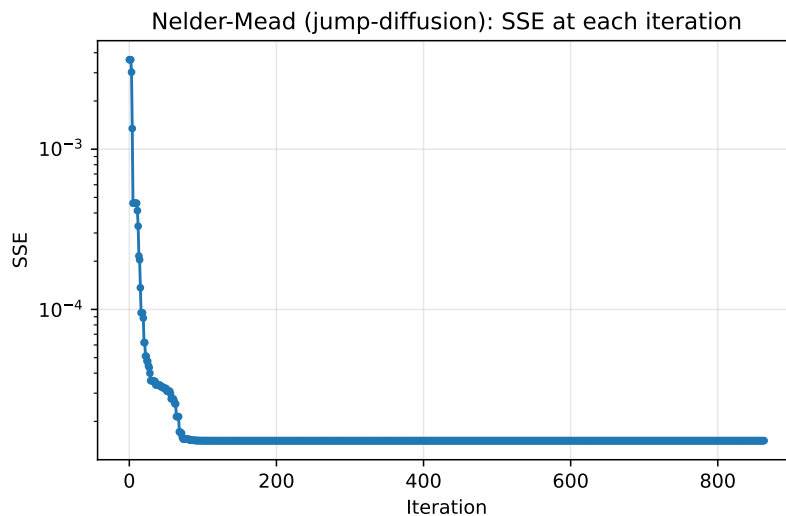


Figure 3: Nelder–Mead (jump-diffusion): SSE at each iteration.

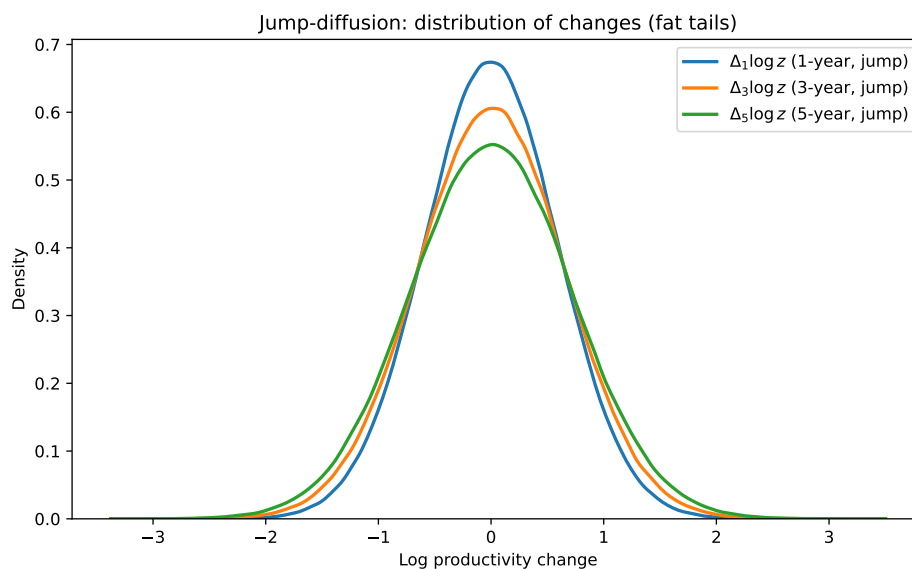


Figure 4: Jump-diffusion: distribution of log-productivity changes (fat tails).

## 2 Root Finding and Nonlinear Equations

Equilibrium conditions in macro models often reduce to solving  $F(x) = 0$  for  $x \in \mathbb{R}^n$ . We summarize univariate and multivariate methods and then apply them to a labor-market equilibrium.

## 2.1 Univariate Methods

**Bisection.** Let  $f: [a, b] \rightarrow \mathbb{R}$  be continuous with  $f(a)f(b) < 0$ . The bisection method constructs a sequence of intervals  $[a_k, b_k]$  with  $f(a_k)f(b_k) < 0$  and  $b_k - a_k = (b - a)/2^k$ . The midpoint  $x_k = (a_k + b_k)/2$  converges to a root  $x^*$  with  $|x_k - x^*| \leq (b - a)/2^{k+1}$ . Convergence is *linear* (one bit of accuracy per iteration) but guaranteed and robust.

**Brent's method.** Brent's method combines bisection with inverse quadratic interpolation. It maintains a bracket where the sign of  $f$  is known and uses the interpolant to propose a new point; if the proposal falls outside the bracket or does not reduce the bracket sufficiently, a bisection step is taken. Under standard assumptions, convergence is *superlinear*; in practice it is the default choice for 1D root-finding when a bracket is available.

**Newton–Raphson.** If  $f$  is  $C^2$  and  $f'(x^*) \neq 0$  at the root  $x^*$ , the iteration  $x_{k+1} = x_k - f(x_k)/f'(x_k)$  converges locally with *quadratic* order:  $|x_{k+1} - x^*| \leq C|x_k - x^*|^2$ . Newton requires the derivative; the secant method replaces  $f'(x_k)$  by a finite-difference approximation and achieves superlinear convergence.

## 2.2 Multivariate Methods

Let  $F: \mathbb{R}^n \rightarrow \mathbb{R}^n$  be continuously differentiable with Jacobian  $J(\mathbf{x}) = \partial F / \partial \mathbf{x}$ .

**Newton's method for systems.** The Newton iteration is  $\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1}F(\mathbf{x}_k)$ . Under standard regularity (non-singular  $J$  at the root, good initial guess), convergence is quadratic. The cost per iteration is  $O(n^3)$  for forming and factoring  $J$ ; when  $J$  is expensive or unavailable, quasi-Newton methods are used.

**Broyden's method (quasi-Newton).** Broyden's method maintains an approximation  $B_k$  to the Jacobian (or its inverse) and updates it with a rank-one correction so that the secant condition  $B_{k+1}(\mathbf{x}_{k+1} - \mathbf{x}_k) = F(\mathbf{x}_{k+1}) - F(\mathbf{x}_k)$  holds. The inverse update is

$$B_{k+1}^{-1} = B_k^{-1} + \frac{(\mathbf{s}_k - B_k^{-1}\mathbf{y}_k)\mathbf{s}_k^\top B_k^{-1}}{\mathbf{s}_k^\top B_k^{-1}\mathbf{y}_k}, \quad \mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k, \quad \mathbf{y}_k = F(\mathbf{x}_{k+1}) - F(\mathbf{x}_k). \quad (10)$$

Convergence is superlinear under standard assumptions. In practice one often uses a line search or backtracking so that  $\|F(\mathbf{x}_{k+1})\|$  decreases; restarts (resetting  $B$  to the identity or a finite-difference Jacobian) improve robustness.

## 2.3 Economic Application: Labor Market Equilibrium

Consider a static economy with a continuum of households indexed by ability  $e \in [1, \bar{e}]$ , distributed according to a truncated Pareto distribution. Each household chooses hours  $h \geq 0$  to maximize CRRA utility in consumption and labor; production is  $Y = L^\eta$  with

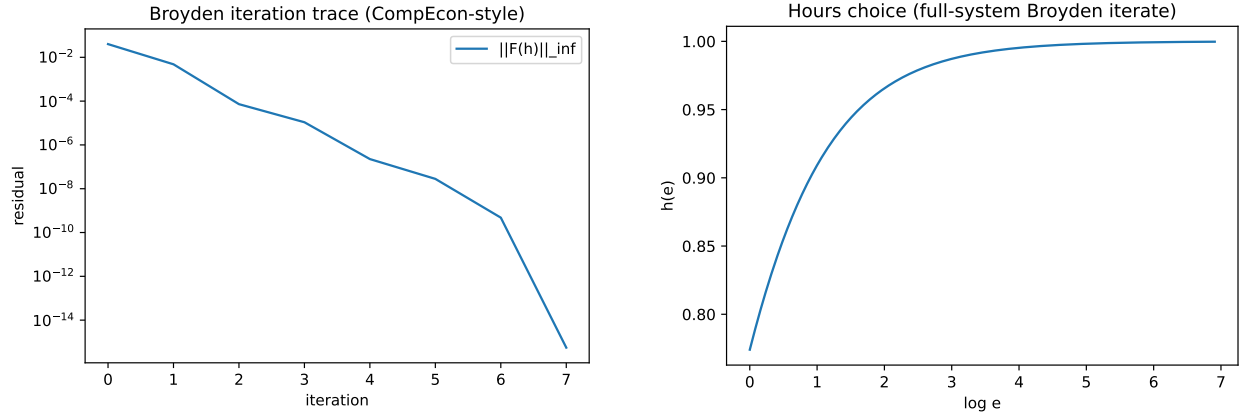
aggregate labor  $L = \int eh(e) dF(e)$ . The wage is  $W = \eta L^{\eta-1}$ ; a linear tax  $\tau$  on labor income is rebated lump-sum. Equilibrium is a wage  $W$ , transfer  $T$ , and schedule  $h(e)$  such that (i) households optimize, (ii) firms maximize profits, and (iii) the goods market clears.

After discretizing the distribution with quadrature weights  $\omega_i$  and nodes  $e_i$ , aggregate labor is  $L(\mathbf{h}) = \sum_i \omega_i e_i h_i$ . The residual for household  $i$ 's first-order condition is

$$F_i(\mathbf{h}) = h_i^\gamma - (1 - \tau)W(\mathbf{h}) e_i c_i(\mathbf{h})^{-\sigma}, \quad (11)$$

with  $W(\mathbf{h}) = \eta L(\mathbf{h})^{\eta-1}$  and  $c_i(\mathbf{h}) = T(\mathbf{h}) + (1 - \tau)W(\mathbf{h})e_i h_i$ . Equilibrium is a root of  $\mathbf{F}(\mathbf{h}) = \mathbf{0}$ . The Jacobian of  $\mathbf{F}$  is dense and model-dependent; Broyden's method (with inverse Jacobian approximation and line search) is a natural choice.

Figure 5 shows the Broyden iteration trace (norm of residual vs. iteration) and the equilibrium hours schedule  $h(e)$  versus  $\log e$ . The full-system Broyden solver converges to a solution that clears the market.



Broyden iteration trace

Equilibrium labor curve  $h(e)$  vs.  $\log e$

Figure 5: Broyden convergence and equilibrium hours (labor supply problem).

### 3 Optimization Methods

Calibration and other numerical tasks require minimizing a scalar objective  $f: \mathbb{R}^n \rightarrow \mathbb{R}$  (e.g., SSE of moments). We summarize derivative-free and gradient-based methods.

#### 3.1 Derivative-Free Methods

**Golden section search (1D).** For a unimodal function  $f: [a, b] \rightarrow \mathbb{R}$ , golden section search maintains an interval containing the minimum and shrinks it by comparing  $f$  at two interior points chosen so that one of them can be reused. The reduction factor per iteration is the golden ratio; convergence is linear. No derivatives are required.

**Nelder–Mead simplex (n-D).** The Nelder–Mead method maintains a simplex of  $n + 1$  points in  $\mathbb{R}^n$  and iteratively reflects, expands, or contracts the worst vertex. It does not use derivatives and is robust to noise and non-smooth objectives, making it standard for simulation-based calibration. Convergence guarantees are weak (no general convergence theorem to a minimizer), but in practice it often works well when the objective is bounded below and the initial simplex is reasonable.

### 3.2 Quasi-Newton Methods

When derivatives are available and the objective is smooth, quasi-Newton methods (e.g., BFGS) approximate the Hessian with a low-rank update and achieve superlinear convergence. BFGS is the dual of Broyden’s method applied to the gradient equation  $\nabla f(\mathbf{x}) = \mathbf{0}$ : the Hessian approximation is updated so that the secant condition for  $\nabla f$  holds. For calibration, the objective is usually simulation-based and non-smooth, so Nelder–Mead is typically preferred over BFGS.

## 4 Euler Equation Methods

Many consumption–savings and portfolio problems are solved by iterating on the Euler equations rather than on the value function. This section summarizes time iteration and Euler residual diagnostics, with an example from a two-period portfolio choice problem.

### 4.1 Time Iteration

Suppose the equilibrium is characterized by a policy function  $c(a, z)$  (or, in the portfolio example, saving and risky shares  $x(m)$ ,  $x_s(m)$ ) satisfying the Euler equations

$$u'(c_t) = \beta \mathbb{E}_t [R_{t+1} u'(c_{t+1})], \quad (12)$$

where  $R_{t+1}$  is the gross return. **Time iteration** (policy function iteration on the Euler equation): guess a policy  $c^{(0)}(a, z)$ ; given  $c^{(k)}$ , compute  $c^{(k+1)}$  by solving the Euler equation at each grid point  $(a, z)$  so that today’s marginal utility equals the discounted expected marginal utility tomorrow. Iterate until the policy stabilizes. This avoids solving for the value function and is often numerically stable.

### 4.2 Euler Residual Diagnostics

At a candidate policy, the **Euler residual** is the difference between the left- and right-hand sides of the Euler equation (often normalized). For example, for the safe asset,

$$\text{residual}_{\text{safe}} = 1 - \beta \mathbb{E} [M_{t+1} R_f], \quad (13)$$

where  $M_{t+1}$  is the stochastic discount factor. A small residual in sup-norm or in  $L^2$  over the state space indicates an accurate solution. Residuals should be checked on a fine grid and, when possible, masked to interior points (where constraints do not bind).

### 4.3 Example: Two-Period Portfolio Choice

Consider a consumer with Epstein–Zin preferences who chooses a saving share  $x$  and a risky share  $x_s$  in period 1; period-2 consumption is  $c_2 = y_2 + (1 + r)xm + \varepsilon_2 x_s m$  with  $(y_2, \varepsilon_2)$  bivariate normal. The Euler equations for the safe and risky assets are

$$1 = \beta \mathbb{E}[M_2(1 + r)], \quad 1 = \beta \mathbb{E}[M_2(1 + r + \varepsilon_2)], \quad (14)$$

where  $M_2$  is the Epstein–Zin SDF. We solve for  $(x(m), x_s(m))$  on a grid of cash-on-hand  $m$  using quadrature for expectations and a derivative-free optimizer for the two-dimensional choice at each  $m$ .

Figure 6 plots the optimal saving share  $x(m)$  and risky share  $x_s(m)$  for two correlation values. Figure 7 shows the Euler residuals for the safe and risky asset; residuals are small in the interior region, confirming numerical accuracy.

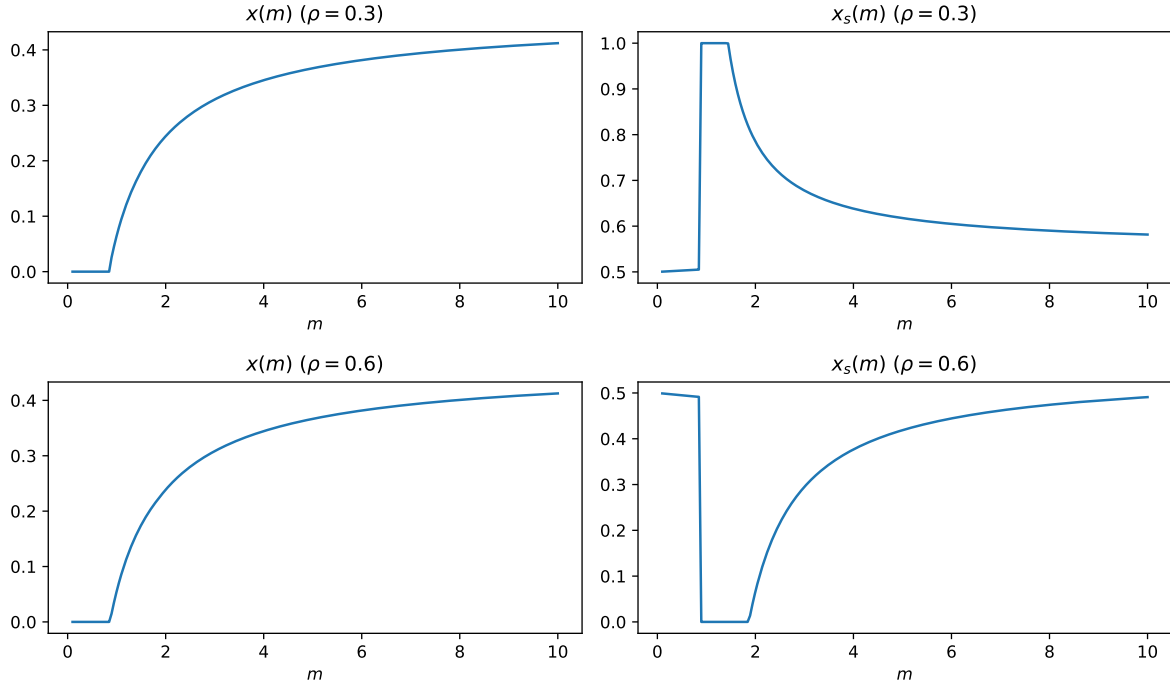


Figure 6: Policy functions: saving share  $x(m)$  and risky share  $x_s(m)$  for two correlation values.



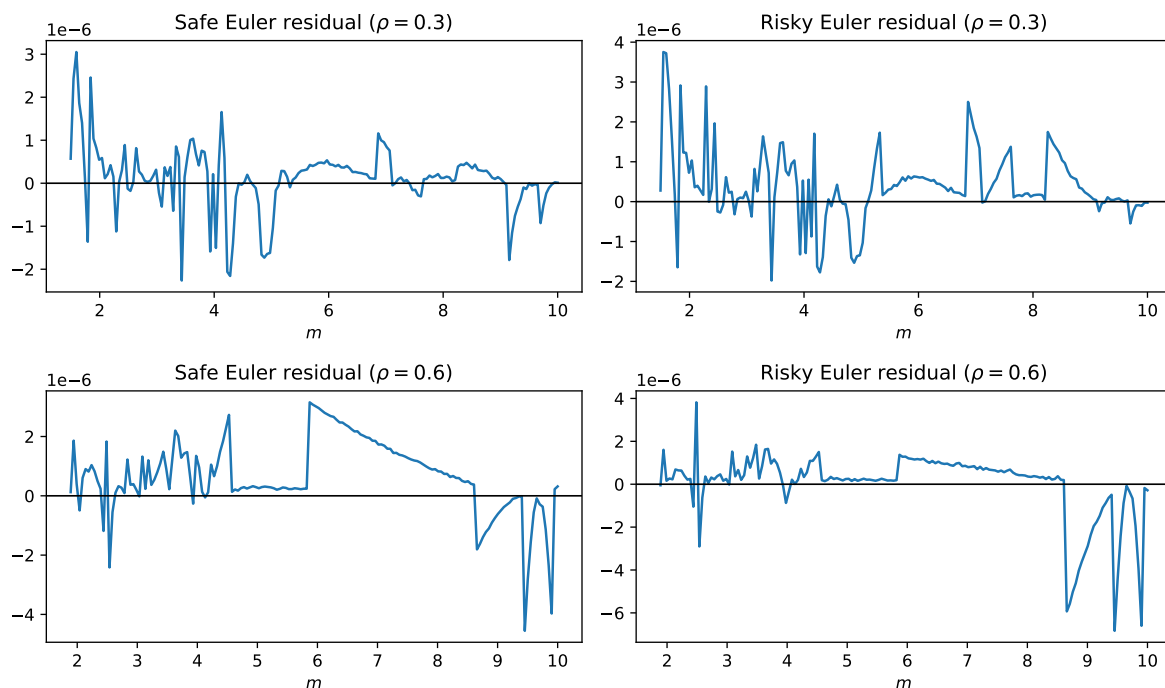


Figure 7: Euler residual diagnostics for the safe and risky asset (interior points).

## References and Cross-Links

For implementation details and code examples, see the **hetmacro Codebook** (Ch. 4: `optimize.py`, calibration workflow). For AI-oriented workflows and calibration playbooks, see **macro\_agents.md**. The figures in this appendix are generated by the Problem Set 1 solutions (Virgiliu's Course, PSet 1).